MINISTRY OF EDUCATION, CULTURE AND RESEARCH OF THE REPUBLIC OF MOLDOVA
Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software Engineering and Automation

Temciuc Adelina, FAF-242

Report

on laboratory work no. 4
Operating Systems
Topic: The xv6 Shell from the Outside

Checked by:
Reiţman Patricia, university assistant
DISA, FCIM, UTM

Chișinău – 2026

AIM OF THE WORK

Purpose of the laboratory work

The purpose of this laboratory work is to understand, by experiment, what a command shell must do and what a full shell such as bash adds on top of that minimum. The xv6 shell (`user/sh.c`, about 500 lines) is used side by side with bash, and a small program that prints its own arguments shows which part of the system expands `*`, removes quotes and splits the command line into words.

The laboratory work aims to:

- update the `sleep` program from Lab 2 for the current xv6, where the system call `sleep()` was renamed to `pause()`;
- run the same command lines in bash and in the xv6 shell and compare redirection, pipes, grouping, sequences, background jobs, globbing, variables, quotes, PATH and history;
- write a program that prints `argc` and `argv` for xv6 and for Linux and use it to find out who expands `*` and removes quotes;
- read the source code of the xv6 shell and find the command kinds it knows, where each kind is executed and how `cd` is handled.

THEORETICAL BACKGROUND

The shell as an ordinary user program

A shell is not part of the kernel. It is an ordinary user program that reads a line, splits it into words, recognises special characters and then asks the kernel to create processes with `fork()` and to load programs with `exec()`. Everything the user sees as "the command line" (pipes, redirection, variables, wildcards) is implemented by the shell itself on top of a few system calls: `fork`, `exec`, `wait`, `open`, `close`, `dup`, `pipe` and `chdir`.

The xv6 shell

In xv6 the whole user interface is the program `user/sh.c`. It parses a line into a tree of commands of five kinds: simple commands (EXEC), redirections (REDIR), pipes (PIPE), lists separated by `;` (LIST) and background commands (BACK); parentheses group commands into a block. The tree is executed by the recursive function `runcmd()` in a child process. The xv6 shell has no variables, no wildcards, no quote handling, no search path and no history.

Arguments, argv and exec

A program receives its command line as an array of strings, `argv`, with `argc` elements; `argv[0]` is the name the program was started with. The kernel does not interpret these strings: `exec(path, argv)` only copies them onto the stack of the new program. Any transformation of the line (wildcard expansion, variable substitution, quote removal, word splitting) must therefore happen in the shell before `exec` is called.

Built-in commands

Some commands cannot be separate programs because they must change the state of the shell process itself. The most important one is `cd`: the current directory is a property of a process, so if `cd` ran in a child process only the child would change directory and the change would be lost when the child exits.

PART 1. FIX SLEEP FOR THE CURRENT XV6

1.1 Checking the name of the system call

In August 2025 the xv6 authors renamed the system call `sleep()` to `pause()`. To see which name the local tree uses, the declaration was searched for in `user/user.h`:

```
cd ~/xv6-riscv
grep -n "pause\|sleep" user/user.h
```

The output was `25:int pause(int);`, so the tree is a current one: only `pause` is declared and the old call `sleep()` from Lab 2 no longer compiles.



Figure 1.1 – The grep result in user.h

1.2 The updated sleep.c

The call in `user/sleep.c` was changed from `sleep()` to `pause()`. The program checks that exactly one argument was given, prints a usage message to standard error (file descriptor 2) otherwise and exits with status 1:

```
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
  if(argc != 2){
    fprintf(2, "usage: sleep <ticks>\n");
    exit(1);
  }
  pause(atoi(argv[1]));   // was sleep() before Aug 2025
  exit(0);
}
```

The line `$U/_sleep\` was added to UPROGS in the Makefile, directly after `$U/_sync\`, starting with a TAB and with nothing after the backslash.

```
student@ubuntu:~$ cd ~/xv6-riscv
student@ubuntu:~/xv6-riscv$ cat user/sleep.c
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
  if(argc != 2){
    fprintf(2, "usage: sleep <ticks>\n");
    exit(1);
  }
  pause(atoi(argv[1]));   // was sleep() before Aug 2025
  exit(0);
}
student@ubuntu:~/xv6-riscv$ grep -n '_sync\|_sleep' Makefile
152:    $U/_sync\
153:    $U/_sleep\
```

Figure 1.2 – The updated sleep.c and the UPROGS line

1.3 Build, boot and test

xv6 was built and started with `make qemu`. At the `$` prompt, `sleep 20` paused the shell for 20 ticks (about two seconds) and then the prompt returned; `sleep` without an argument printed the usage message.

```
student@ubuntu:~/xv6-riscv$ make qemu
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M -smp
3 -nographic -global virtio-mmio.force-legacy=false -drive file=fs.img,if=none,f
ormat=raw,id=x0 -device virtio-blk-device,drive=x0,bus=virtio-mmio-bus.0

xv6 kernel is booting

hart 1 starting
hart 2 starting
init: starting sh
$ sleep 20
$ sleep
usage: sleep <ticks>
$
```

Figure 1.3 – sleep running in xv6, with and without an argument

PART 2. BASH VERSUS THE XV6 SHELL

2.1 Running the commands in both shells

Every command from the table was run twice: once in bash, in an empty test folder that contains only the file README (`mkdir ~/shtest && cd ~/shtest && touch README`), and once at the xv6 `$` prompt after `make qemu`. The results are summarised in table 2.1.

Table 2.1 – The same commands in bash and in the xv6 shell

Command	Tests	bash	xv6	Why
`echo hello > f ; cat < f`	redirection	hello	hello	Both shells open the file and connect it to stdout or stdin before exec.
`ls | grep READ`	pipe	README	README 2 2 2441	Both create a pipe; xv6 ls also prints type, inode and size.
`(echo a; echo b) | wc`	grouping, list	2 2 4	2 2 4	Both support ( ) and ;. The numbers are the same, only the spacing of wc differs.
`echo one ; echo two`	sequence	one two	one two	Both run the commands one after another.
`echo bg &`	background	[1] 2672 bg [1]+ Done	bg$ (output mixed with the prompt)	Both run it without waiting; only bash has job control, so only bash prints the job number and "Done".
`ls *`	globbing	f README	ls: cannot open *	bash replaces * with the file names; xv6 passes the character * to ls unchanged.
`echo $HOME`	variables	/home/student	$HOME	xv6 has no variables, so $HOME is an ordinary word.
`echo "a b"`	quotes	a b	"a b"	bash removes the quotes and keeps one word; xv6 keeps the quotes and splits it into two words.
`mkdir d ; cd d ; ls ..`	PATH	d f README	exec cd failed, then the listing of /	xv6 treats cd as a built-in only at the start of a line; here it tries to exec a program cd, which does not exist.
`arrow up, Tab, Ctrl+R`	history, completion	recalls, completes and searches commands	exec arrow failed	xv6 has no line editor, history or completion; the text is run as a command.

In bash, the up arrow recalled the previous command, Tab completed `../RE` to `../README` and Ctrl+R opened the reverse history search, shown at the bottom of figure 2.1.

```
student@ubuntu:~$ mkdir ~/shtest && cd ~/shtest && touch README
student@ubuntu:~/shtest$ echo hello > f ; cat < f
hello
student@ubuntu:~/shtest$ ls | grep READ
README
student@ubuntu:~/shtest$ (echo a; echo b) | wc
      2       2       4
student@ubuntu:~/shtest$ echo one ; echo two
one
two
student@ubuntu:~/shtest$ echo bg &
[1] 2672
bg
student@ubuntu:~/shtest$ ls *
f  README
[1]+  Done                    echo bg
student@ubuntu:~/shtest$ echo $HOME
/home/student
student@ubuntu:~/shtest$ echo "a b"
a b
student@ubuntu:~/shtest$ mkdir d ; cd d ; ls ..
d  f  README
student@ubuntu:~/shtest/d$ ls ../README
../README
(reverse-i-search)`hel': echo hello > f ; cat < f
```

Figure 2.1 – The commands run in bash

```
init: starting sh
$ echo hello > f ; cat < f
hello
$ ls | grep READ
README          2 2 2441
$ (echo a; echo b) | wc
2 2 4
$ echo one ; echo two
one
two
$ echo bg &
bg$
ls *
ls: cannot open *
$ echo $HOME
$HOME
$ echo "a b"
"a b"
$
```

Figure 2.2 – The same commands run in xv6

Figure 2.3 shows the end of the xv6 run. Because `cd` was not at the beginning of the line, xv6 printed `exec cd failed`, stayed in `/` and `ls ..` listed the root directory, where the new file `f` (6 bytes, "hello" and a newline) and the empty directory `d` (32 bytes, the entries `.` and `..`) appear. The keys line was executed as the program `arrow`.

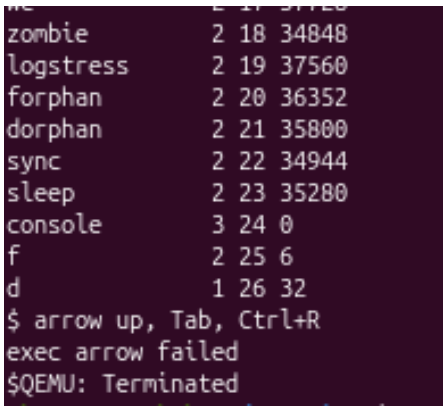


Figure 2.3 – The end of the xv6 run: ls .. and the history keys

Then `cd d` was typed alone on a line, so the built-in worked, and `ls` was tried again inside `d`. It failed with `exec ls failed`. Giving the path of the program, `/ls` or `../ls`, made it work.

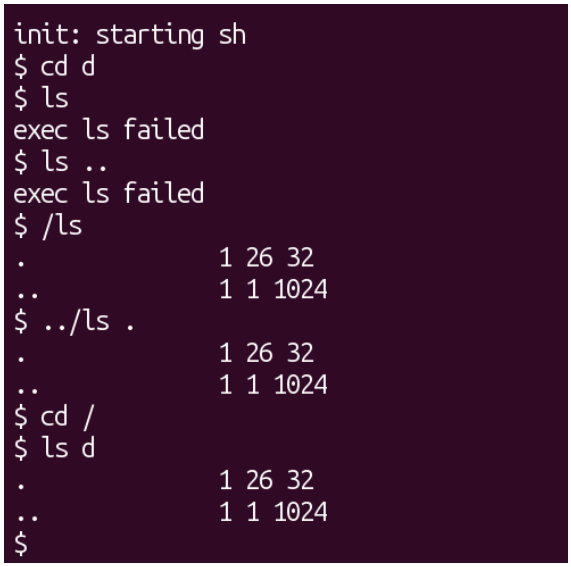


Figure 2.4 – ls failing after cd d, and the working alternative

2.2 Answers to the questions

1. Which features do both shells have, and which ones only bash has?

Both shells can run programs with arguments, redirect input and output with <, > and >>, connect commands with pipes `|`, run sequences with `;`, group commands with parentheses, start commands in the background with `&` and change the directory with the built-in `cd`.

Only bash has wildcards (globbing), variables and `$` expansion, quote removal, a search path (PATH), job control (job numbers, `jobs`, `fg`, "Done" messages), `&&` and `||`, command history, Tab completion and history search, and a full scripting language with `if`, `for` and functions.

2. Why does ls stop working after cd d, and how can it be run anyway?

The xv6 programs live in the root directory `/`, and the xv6 shell has no PATH: it calls `exec` with exactly the name that was typed. `exec("ls")` is a relative path, so the kernel looks for `ls` in

the current directory. In `/` it is found, but in `/d` there is no file `ls`, so `exec` fails. The program can still be run by giving its location: `/ls` (absolute path) or `../ls` (relative path), as shown in figure 2.4.

3. Which missing feature would you miss most, and which would be easiest to add?

In daily work I would miss history and Tab completion the most, because they save a lot of typing, together with wildcards. The easiest feature to add is a simple PATH: if `exec(argv[0])` fails and the name does not contain a `/`, the shell can try again with `"/"` in front of the name. That is only a few lines in the EXEC case of `runcmd()` and would fix the `ls` problem from question 2.

PART 3. WHO EXPANDS *? A PROGRAM THAT PRINTS ITS ARGV

3.1 The args program for xv6

The program `user/args.c` prints the number of arguments and every argument between square brackets, so that spaces and quotes inside an argument are visible. The line `$U/_args\` was added to UPROGS after `$U/_sleep\`.

```
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
  printf("argc = %d\n", argc);
  for(int i = 0; i < argc; i++)
    printf("argv[%d] = [%s]\n", i, argv[i]);
  exit(0);
}
```

```
student@ubuntu:~$ cd ~/xv6-riscv
student@ubuntu:~/xv6-riscv$ cat user/args.c
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
  printf("argc = %d\n", argc);
  for(int i = 0; i < argc; i++)
    printf("argv[%d] = [%s]\n", i, argv[i]);
  exit(0);
}
student@ubuntu:~/xv6-riscv$ grep -n '_sleep\|_args' Makefile
153:    $U/_sleep\
154:    $U/_args\
```

Figure 3.1 – args.c and the UPROGS line

3.2 The same program for Linux

For Linux, the file `args_linux.c` includes `<stdio.h>` instead of the two xv6 headers and ends with `return 0` instead of `exit(0)`. It was compiled with `gcc -o args args_linux.c` in the folder `~/lab4`, which also contains the files `args.c`, `args_linux.c` and `sleep.c`.

```
#include <stdio.h>

int
main(int argc, char *argv[])
{
  printf("argc = %d\n", argc);
  for(int i = 0; i < argc; i++)
```

```
    printf("argv[%d] = [%s]\n", i, argv[i]);
  return 0;
}
```

3.3 Running the same lines in both shells

The following lines were run in xv6 and, with `./args`, in bash:

```
args one   two   three
args *.c "x y"
args $HOME
args 'single quoted'
```

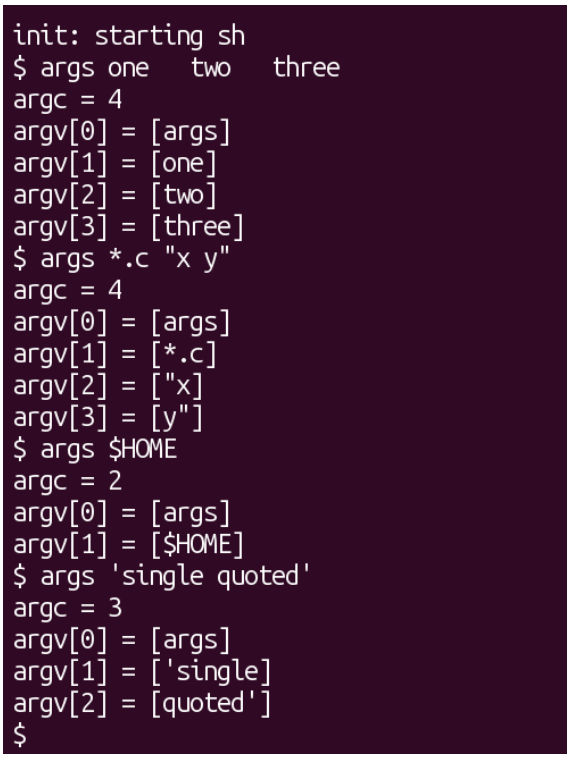


Figure 3.2 – args output in xv6

```
student@ubuntu:~$ cd ~/lab4
student@ubuntu:~/lab4$ gcc -o args args_linux.c
student@ubuntu:~/lab4$ ls
args  args.c  args_linux.c  sleep.c
student@ubuntu:~/lab4$ ./args one   two   three
argc = 4
argv[0] = [./args]
argv[1] = [one]
argv[2] = [two]
argv[3] = [three]
student@ubuntu:~/lab4$ ./args *.c "x y"
argc = 5
argv[0] = [./args]
argv[1] = [args.c]
argv[2] = [args_linux.c]
argv[3] = [sleep.c]
argv[4] = [x y]
student@ubuntu:~/lab4$ ./args $HOME
argc = 2
argv[0] = [./args]
argv[1] = [/home/student]
student@ubuntu:~/lab4$ ./args 'single quoted'
argc = 2
argv[0] = [./args]
argv[1] = [single quoted]
```

Figure 3.3 – args_linux output in bash

3.4 Answers to the questions

1. Both outputs for args *.c "x y". How many arguments did each program receive?

In xv6:

```
argc = 4
argv[0] = [args]
argv[1] = [*.c]
argv[2] = ["x]
argv[3] = [y"]
```

In bash:

```
argc = 5
argv[0] = [./args]
argv[1] = [args.c]
argv[2] = [args_linux.c]
argv[3] = [sleep.c]
argv[4] = [x y]
```

In xv6 the program received 4 arguments: `*.c` unchanged, and `"x` and `y"` as two separate words with the quote characters still in them. In bash it received 5 arguments: the three `.c` files of the folder instead of `*.c`, and `x y` as one argument, without quotes.

2. Who turns *.c into file names and removes the quotes?

The shell. The program is the same in both cases and only prints what it receives, and the kernel only copies the `argv` array given to `exec`. The difference comes only from the shell: bash

expands `*.c`, substitutes `$HOME` and removes quotes before calling `exec`, while the xv6 shell does none of this, so the program sees the raw words.

3. What happened to the extra spaces in one two three?

They disappeared in both shells. The shell uses spaces only as separators between words, so any number of spaces between two words produces just one boundary, and the program received exactly three arguments: `one`, `two` and `three`. To keep the spaces they must be quoted in bash (`"one  two"`); in xv6 this is impossible, because it does not handle quotes.

PART 4. READ THE SHELL

4.1 Commands executed

Inside `~/xv6-riscv` the following commands were run:

```
wc -l user/sh.c
grep -n "#define" user/sh.c        # the kinds of commands
grep -n "case " user/sh.c          # where each kind is run
grep -n "cd" user/sh.c | head
```

The file `user/sh.c` has 499 lines. Lines 8–12 define the five command kinds.

```
student@ubuntu:~$ cd ~/xv6-riscv
student@ubuntu:~/xv6-riscv$ wc -l user/sh.c
499 user/sh.c
student@ubuntu:~/xv6-riscv$ grep -n "#define" user/sh.c
8:#define EXEC  1
9:#define REDIR 2
10:#define PIPE  3
11:#define LIST  4
12:#define BACK  5
14:#define MAXARGS 10
```

Figure 4.1 – The command kinds defined in sh.c

The cases on lines 75–125 are in `runcmd()`, which executes each kind. Lines 284–294 are in `gettoken()`, which recognises the special characters, and lines 469–493 are in `nulterminate()`, which finishes the parsed tree.

```
student@ubuntu:~$ cd ~/xv6-riscv
student@ubuntu:~/xv6-riscv$ grep -n "case " user/sh.c
75:  case EXEC:
83:  case REDIR:
93:  case LIST:
101:  case PIPE:
125:  case BACK:
284:  case 0:
286:  case '|':
287:  case '(':
288:  case ')':
289:  case ';':
290:  case '&':
291:  case '<':
294:  case '>':
391:    case '<':
394:    case '>':
397:    case '+': // >>
469:  case EXEC:
475:  case REDIR:
481:  case PIPE:
487:  case LIST:
493:  case BACK:
```

Figure 4.2 – Where each kind is run

`grep -n "cd"` finds only the error message on line 170, because the code tests the characters `c` and `d` separately. Searching for `chdir` shows the whole block in `main()`.

```
student@ubuntu:~$ cd ~/xv6-riscv
student@ubuntu:~/xv6-riscv$ grep -n "cd" user/sh.c | head
170:        fprintf(2, "cannot cd %s\n", cmd + 3);
student@ubuntu:~/xv6-riscv$ grep -n -B4 -A3 "chdir" user/sh.c
165-      continue;
166-    if (cmd[0] == 'c' && cmd[1] == 'd' && cmd[2] == ' ') {
167-      // Chdir must be called by the parent, not the child.
168-      cmd[strlen(cmd) - 1] = 0; // chop \n
169:      if (chdir(cmd + 3) < 0)
170-        fprintf(2, "cannot cd %s\n", cmd + 3);
171-    } else {
172-      if (fork1() == 0)
```

Figure 4.3 – The code that handles cd

Finally, `sh.c` was searched for any code that handles `*` or `$`. The search printed nothing; the only special characters the shell knows are listed in the arrays `whitespace` and `symbols`.

```
student@ubuntu:~$ cd ~/xv6-riscv
student@ubuntu:~/xv6-riscv$ grep -n "'\*'\|'\$'\|glob\|getenv" user/sh.c
student@ubuntu:~/xv6-riscv$ grep -n 'symbols\[\]\|whitespace\[\]' user/sh.c
268:char whitespace[] = " \t\r\n\v";
269:char symbols[] = "<|>&;()";
```

Figure 4.4 – No code for * or $ in sh.c

4.2 Answers to the questions

1. The five kinds of command and the character that produces each

- EXEC (1) – a simple command, a program name with its arguments; it needs no special character;
- REDIR (2) – redirection, produced by <, > or >>;
- PIPE (3) – a pipe, produced by `|`;
- LIST (4) – a sequence, produced by `;`;
- BACK (5) – a background command, produced by `&`.

Parentheses `( )` only group commands into a block; they do not create a sixth kind.

2. Where is cd handled, and why is it not run as a separate program?

`cd` is handled in `main()`, on lines 166–170, before the shell forks: if the line starts with the characters `c`, `d` and a space, the shell calls `chdir(cmd + 3)` itself and prints `cannot cd` if it fails. It is not a separate program because every other command runs in a child process created by `fork()`. A child that called `chdir` would change only its own current directory, and the change would be lost when the child exits, so the shell would stay where it was. The comment in the code

says it directly: "Chdir must be called by the parent, not the child". This is also why `cd` in the middle of `mkdir d ; cd d ; ls ..` failed: there it is parsed as a normal EXEC command.

3. Is there any code that handles * or $? What does that explain about Part 2?

No. The tokenizer only knows the whitespace characters `" \t\r\n\v"` and the symbols `<|>&;()`; there is no code for `*`, `$`, quotes or PATH. Every other character is just part of a word. This explains the results of Part 2 and Part 3: `ls *` passed the character `*` to `ls`, `echo $HOME` printed `$HOME`, `echo "a b"` kept the quotes and split the text into two words, and `ls` could not be found outside `/`.

CONCLUSION

In this laboratory work the xv6 shell was studied from the outside, by using it side by side with bash and by reading its source code.

In the first part, the `sleep` program from Lab 2 was updated for the current xv6, where the system call is called `pause()`. After adding it to UPROGS it built and worked: `sleep 20` paused the shell and `sleep` without an argument printed the usage message.

In the second part, the same commands were run in both shells. Both support redirection, pipes, grouping, sequences, background commands and `cd`, which are exactly the features that need only `fork`, `exec`, `pipe`, `dup` and `chdir`. Everything that is about text processing of the command line (wildcards, variables, quotes, PATH, history and completion) exists only in bash. The xv6 shell also showed that `cd` is a built-in only at the beginning of a line, and that without PATH a program in `/` must be called with its full path from any other directory.

In the third part, the `args` program proved that the program and the kernel do not change the arguments: for `args *.c "x y"` the xv6 program received 4 raw words, while the bash version received 5 arguments with the file names expanded and the quotes removed. The shell alone is responsible for expansion, quote removal and word splitting.

In the fourth part, reading `user/sh.c` (499 lines) showed the five command kinds EXEC, REDIR, PIPE, LIST and BACK, the function `runcmd()` that executes them, the special handling of `cd` in the parent process and the absence of any code for `*` or `$`, which explains all the differences observed in Part 2.

REFERENCES

1. Cox R., Kaashoek F., Morris R. xv6: a simple, Unix-like teaching operating system. MIT, 2024. [online]. Available: https://pdos.csail.mit.edu/6.1810/2024/xv6/book-riscv-rev4.pdf
2. mit-pdos/xv6-riscv – source code. [online]. Available: https://github.com/mit-pdos/xv6-riscv
3. bash(1) – Linux manual page. [online]. Available: https://man7.org/linux/man-pages/man1/bash.1.html
4. GNU Bash Reference Manual. Free Software Foundation. [online]. Available: https://www.gnu.org/software/bash/manual/bash.html